Quiz App Specification Document

1. Project Overview & Scope

This Specification Document details the functional, technical, and architectural requirements for a full-stack, AI-assisted Quiz Web Application. Built as part of a high school web development project, the application provides interactive quiz experiences for students (players) and comprehensive administrative tools for instructors (teachers/admins). The system enforces strict security standards, data persistence across app restarts, dynamic leaderboards, and administrative bulk-management tools.

2. Mandatory Functional Requirements

2.1 Quiz & Question Requirements

- **Quiz Count & Questions:** The application includes at least five distinct, fully functioning quizzes, each containing a minimum of ten questions.
- **Question Format:** Every question provides exactly four answer choices with strictly one correct answer designated.
- **Execution Flow:** Registered users can browse available quizzes, complete a quiz session, view real-time score results and percentage scores upon completion, and retry quizzes at any time.

2.2 Authentication & User Roles

- **Account Lifecycle:** Complete support for user registration, user login, and secure session logout.
- **Role-Based Access Control (RBAC):** Explicit role separation between **Player** (student) and **Teacher/Admin** permissions.
- **Password Security:** Passwords must never be stored as plain text. All user credentials must be salted and hashed prior to persistence.

2.3 Attempt Tracking & User History

- **Attempt History:** Users can access a personal history log showing all their completed quiz attempts.
- **Data Recording:** Every completed attempt—including repeat/retry attempts—is recorded in the database with the associated user, quiz ID, score, total question count, and completion timestamp.
- **Incomplete Attempts:** Abandoned or incomplete quiz attempts are excluded from history and leaderboard calculations.

2.4 Leaderboard System

- **Overall Leaderboard:** Ranks users based on total completed quiz attempts.

- **Per-Quiz Leaderboard:** Ranks users for a specific quiz by their highest score achieved, displaying the total attempt count as secondary context.
- **Privacy Protection:** All public leaderboards display user-defined usernames only, strictly omitting user email addresses.

2.5 Teacher & Admin Features

- **Interactive Question Editor:** Teachers/admins can create, edit, update, and remove questions, answer choices, correct flags, and quiz assignments via a web dashboard.
- **CSV Upload & Validation:** Instructors can upload questions via a standardized CSV template. Uploads undergo front-end and back-end parsing, showing a preview table with line-by-line error flags before committing any data to the database.

2.6 Additional Functional Features

In addition to core requirements, two functional features are specified:

1. **Timed Quiz Mode (Countdown Timer):** Optional quiz timer enforcing a per-question or per-quiz time limit, automatically submitting answers when time expires.
2. **Detailed Question Review & Explanations:** Post-quiz review screen allowing users to see their selected choices alongside correct choices and detailed explanations for each question.

3. Technical Architecture & Tech Stack

Layer / Component	Technology / Standard	Role & Description
Frontend	Plain HTML5, CSS3, JavaScript (ES6+)	Responsive UI, dynamic DOM manipulation, client-side validation, and fetch API calls.
Server / API	Node.js 24 LTS & Express 5	RESTful API endpoints serving static web assets and JSON dynamic routes.
Package Manager	npm	Dependency management with committed package-lock.json for reproducible builds.
Database Driver	Replit PostgreSQL (pg library)	Relational SQL database using parameterized queries to protect against SQL injection.
AI Assistant	Mistral Code	AI-assisted code generation,

Layer / Component	Technology / Standard	Role & Description
		refactoring, and code review.
Version Control	GitHub	Source code repository, documentation management, and revision history.
Environments	Replit Workspace & Replit Deployment	Replit workspace serves as QA; Replit Deployment serves as Production with isolated production DB.

4. Database Schema Structure

The relational database schema consists of five main tables designed with foreign key constraints and indexed lookup fields:

```
users (
  id SERIAL PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(20) DEFAULT 'player',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

quizzes (
  id SERIAL PRIMARY KEY,
  title VARCHAR(150) NOT NULL,
  description TEXT,
  created_by INT REFERENCES users(id),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

questions (
  id SERIAL PRIMARY KEY,
  quiz_id INT REFERENCES quizzes(id) ON DELETE CASCADE,
  question_text TEXT NOT NULL,
  explanation TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

choices (
  id SERIAL PRIMARY KEY,
  question_id INT REFERENCES questions(id) ON DELETE CASCADE,
  choice_text TEXT NOT NULL,
```

```
  is_correct BOOLEAN DEFAULT FALSE
);

quiz_attempts (
  id SERIAL PRIMARY KEY,
  user_id INT REFERENCES users(id) ON DELETE CASCADE,
  quiz_id INT REFERENCES quizzes(id) ON DELETE CASCADE,
  score INT NOT NULL,
  total_questions INT NOT NULL,
  percentage NUMERIC(5,2) NOT NULL,
  is_completed BOOLEAN DEFAULT TRUE,
  completed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5. Development Roadmap (Five Phases)

Phase 1: Environment Setup & Database Foundation

- Initialize Node.js 24 LTS and Express 5 server repository on GitHub with package-lock.json.
- Provision Replit PostgreSQL database and run SQL migration scripts to construct database tables.
- Configure database pool using pg module with parameterized SQL handlers.

Phase 2: Authentication & Authorization

- Implement user registration and login endpoints utilizing secure password hashing (bcrypt).
- Establish session state and role verification middleware for Player vs. Admin access.
- Build user registration UI and secure login views.

Phase 3: Quiz Execution & Scoring Engine

- Construct REST API endpoints to fetch active quizzes, questions, and four choices per question.
- Develop responsive frontend quiz player with selection states and timer functionality.
- Build backend quiz evaluation endpoint to calculate score, save completed attempt to quiz_attempts, and return percentage result.

Phase 4: Leaderboards, History, & Custom Features

- Develop API query routes for Overall Leaderboard (by completed attempt count) and Quiz Leaderboards (by highest score).
- Build user history view rendering past attempts with dates, scores, and percentages.
- Implement custom post-quiz answer explanation and breakdown view.

Phase 5: Admin Dashboard, CSV Import, & Production Deployment

- Construct Admin Question Editor web interface for manual question management.
- Develop CSV file parser with client/server validation preview identifying invalid records prior to import.
- Conduct end-to-end QA testing in Replit workspace environment and deploy to Replit Production with isolated database.